

```
show dbs
db
use test
show collections
```

```
// Inserts
```

```
var nuevo_item = { "item": "camiseta", "qty": 100, "price": 19.5, "tag": "algodon", "size_h": 28,
"size_w": 35.5, "size_uom": "cm" }
db.inventory.insertOne( nuevo_item )
db.inventory.find()
```

```
var array_items= [
{ "item": "camiseta", "qty": 100, "price": 19.5, "tag": "algodon", "size_h": 28, "size_w": 35.5,
"size_uom": "cm" },
{ "item": "camiseta", "qty": 100, "price": 22.5, "tag": "algodon", "size_h": 56, "size_w": 47,
"size_uom": "pu" },
{ "item": "medias", "qty": 10, "price": 1.5, "tag": "licra", "size_h": 2, "size_w": 3, "size_uom":
"cm" },
{ "item": "corbata", "qty": 6, "price": 5.5, "tag": "algodon", "size_h": 5, "size_w": 4, "size_uom":
"cm" },
{ "item": "camiseta", "qty": 78, "price": 10.5, "tag": "carton", "size_h": 4, "size_w": 1,
"size_uom": "cm" }
]
db.inventory.insertMany( array_items )
```

```
db.inventory.find()
```

```
// Finds
```

```
var query= { "item": "camiseta" }
db.inventory.find( query )
```

```
var query= { "item": "camiseta", "qty": 78 }
db.inventory.find( query )
```

```
// Sin resultados
```

```
var query= { "item": "camiseta", "qty": 78, "nu": 9 }
db.inventory.find( query )
```

```
// Proyecciones
```

```
var query= { "item": "camiseta", "qty": 78}
var proyeccion = { "qty": 1 }
db.inventory.find( query, proyeccion )
```

```
var query= { }
var proyeccion = { "_id": 0, "qty": 1 }
db.inventory.find( query, proyeccion )
```

```
var query= { }
```

```
var proyeccion = { "_id": 0, "qty": 0 }  
db.inventory.find( query, proyeccion )
```

```
// Update  
var query= { }  
var operacion= { $set: { "campo": "borrar" } }  
db.inventory.updateMany(query,operacion)
```

```
db.inventory.find()
```

```
var query= { }  
var operacion= { $unset: { "campo": "borrar" } }  
db.inventory.updateMany(query,operacion)  
db.inventory.find()
```

```
// Delete  
var reg= { "para": "borrar" }  
db.inventory.insertOne(reg)  
db.inventory.find(reg)
```

```
db.inventory.deleteMany(reg)  
db.inventory.find(reg)
```

```
// Filtros avanzados  
// selectores  
var query= { "qty": { $gt: 78 } }  
db.inventory.find( query )
```

```
var query= { "qty": { $lt: 78 } }  
db.inventory.find( query )
```

```
var query= { "qty": { $ne: 10 } }  
db.inventory.find( query )
```

```
db.inventory.find()
```

```
// lógicos
```

```
var query1= { "qty": { $ne: 10 } }  
var query2= { "qty": { $lt: 70 } }  
var logic= { $and: [ query1, query2 ] }  
db.inventory.find( logic )
```

```
var query1= { "qty": { $eq: 100 } }  
var query2= { "qty": { $lt: 50 } }  
var logic= { $nor: [ query1, query2 ] }  
db.inventory.find( logic )
```

```
var logic= { $or: [ query1, query2 ] }  
db.inventory.find( logic )
```

```
//elementos
```

```
var query = { "qty": { $exists: true } }  
db.inventory.find( query )
```

```
var query = { "qty": { $exists: false } }  
db.inventory.find( query )
```

```
var query = { "noexiste": { $exists: false } }  
var operacion= { $set: { "noexiste": null } }  
db.inventory.updateOne( query, operacion )
```

```
var query = { "noexiste": { $exists: true } }  
db.inventory.find( query )
```

```
var query = { "qty": { $type: "double" } }  
db.inventory.find( query )
```

```
var query = { "noexiste": { $type: "null" } }  
db.inventory.find( query )
```

```
// Documento Embebido
```

```
var array_items= [  
  { "item": "camiseta", "qty": 100, "price": 19.5, "tag": "algodon", "size": { "h": 28, "w": 35.5,  
    "uom": "cm" } } ,  
  { "item": "camiseta", "qty": 100, "price": 22.5, "tag": "algodon", "size": { "h": 56, "w": 47, "uom":  
    "pu" } } ,  
  { "item": "medias", "qty": 10, "price": 1.5, "tag": "licra", "size": { "h": 2, "w": 3, "uom": "cm" } } ,  
  { "item": "corbata", "qty": 6, "price": 5.5, "tag": "algodon", "size": { "h": 5, "w": 4, "uom": "cm" }  
  },  
  { "item": "camiseta", "qty": 78, "price": 10.5, "tag": "carton", "size": { "h": 4, "w": 1, "uom": "cm"  
  } }  
]  
db.inventoryE.insertMany( array_items )
```

```
db.inventoryE.find()
```

```
// Sobre elemento
```

```
var query = { "size.h": 56 }  
db.inventoryE.find( query )
```

```
// Sobre varios elementos
```

```
var query = { "size.h": { $lt: 15 }, "size.uom": "cm" }  
db.inventoryE.find( query )
```

```

// Array
var array_items= [
{ "item": "camiseta", "qty": 100, "price": 19.5, "tag": ["algodon","fino"], "size": { "h": 28, "w": 35.5, "uom": "cm" } },
{ "item": "camiseta", "qty": 100, "price": 22.5, "tag": ["algodon","fino"], "size": { "h": 56, "w": 47, "uom": "pu" } },
{ "item": "medias", "qty": 10, "price": 1.5, "tag": ["licra"], "size": { "h": 2, "w": 3, "uom": "cm" } },
{ "item": "corbata", "qty": 6, "price": 5.5, "tag": ["algodon","seda"], "size": { "h": 5, "w": 4, "uom": "cm" } },
{ "item": "camiseta", "qty": 78, "price": 10.5, "tag": ["carton","piedra"], "size": { "h": 4, "w": 1, "uom": "cm" } }
]
db.inventoryA.insertMany( array_items )

db.inventoryA.find()

// Elementos de array en el orden, número de elementos y valor exacto
var query = { tag: ["algodon", "seda"] }
db.inventoryA.find( query )

var query = { tag: ["seda", "algodon"] }
db.inventoryA.find( query )

// Elementos de array que incluye los valores (sin orden)
var query = { tag: { $all: ["seda", "algodon"] } }
db.inventoryA.find( query )

// Elementos de array que satisface al menos un valor o condición
var query = { tag: "seda" }
db.inventoryA.find( query )

// Elementos de array que satisface al menos un valor o condición de todas ellas
var query = { tag: { $in: ["seda","licra"] } }
db.inventoryA.find( query )

// Elementos de array que satisface todas las condiciones
var subquery= { $nin: ["seda","licra"] }
var query={ tag: { $elemMatch: subquery } }
db.inventoryA.find( query )

// Un elementos del array
var query={ "tag.0": "algodon" }
db.inventoryA.find( query )

// Consulta sobre el número de elementos del array
var query={ "tag": { $size: 2 } }
db.inventoryA.find( query )

```

```
// Array Subdocuments
var array_items= [
{ "item": "camiseta", "qty": 100, "price": 19.5, "tag": ["algodon","fino"], "size": [{ "h": 28, "w": 35.5, "uom": "cm" }, { "h": 56, "w": 47, "uom": "pu" } ] },
{ "item": "medias", "qty": 10, "price": 1.5, "tag": ["licra"], "size": [ { "h": 2, "w": 3, "uom": "cm" } ] },
{ "item": "corbata", "qty": 6, "price": 5.5, "tag": ["algodon","seda"], "size": [ { "h": 5, "w": 4, "uom": "cm" } ] },
{ "item": "camiseta", "qty": 78, "price": 10.5, "tag": ["carton","piedra"], "size": [ { "h": 4, "w": 1, "uom": "cm" } ] }
]
db.inventoryAE.insertMany( array_items )
```

```
db.inventoryAE.find()
// Consulta sobre todos los documentos de un array, donde al menos un documento debe
ser exactamente igual, valores, orden y número de campos/claves
var query={size: { "h": 28, "w": 35.5, "uom": "cm" } }
db.inventoryAE.find( query )
```

```
var query={size: { "h": 28, "w": 35.5 } }
db.inventoryAE.find( query )
```

```
var query={size: { "w": 35.5, "uom": "cm", "h": 28 } }
db.inventoryAE.find( query )
```

```
// Consulta sobre todos los documentos de un array, donde al menos un documento debe
tener el campo/clave a filtrar pero no es necesario especificar el orden del mismo
var query={ "size.w": 35.5 }
db.inventoryAE.find( query )
```

```
// Consulta sobre un documento del array (posición específica) , donde debe tener el
campo/clave a filtrar sin especificar el orden
var query={ "size.0.w": 35.5 }
db.inventoryAE.find( query )
```

```
var query={ "size.1.w": 35.5 }
db.inventoryAE.find( query )
```

```
// Consulta sobre todos los documentos de un array, donde al menos un documento
satisface todas las condiciones
var query={ size: { $elemMatch: { "w": 35.5, "uom": "cm" } } }
db.inventoryAE.find( query )
```

```
// Consulta sobre todos los documentos de un array, donde cada condición se debe
satisfacer aunque no sean sobre el mismo documento dentro del array
var query={ "size.h": { $gt: 30, $lte: 56 } }
db.inventoryAE.find( query )
```

// La próxima query no devuelve nada, no existe dentro del array size documentos que tengan el campo h para algún caso menor igual que 27 y mayor o igual que 55 (el ejemplo de antes tiene 57 y 28)

```
var query={ "size.h": { $gt: 55, $lte: 27 } }  
db.inventoryAE.find( query )
```

// Consulta sobre todos los documentos de un array, donde al menos un documento satisface todas las condiciones

// No devuelve nada

```
var query={ "size": { $elemMatch: { "h": { $gt: 30, $lte: 55 } } } }  
db.inventoryAE.find( query )
```

// Update

// \$addToSet

```
var query= { }  
var operacion= { $addToSet: { "tag": "borrar" } }  
db.inventoryA.updateMany(query,operacion)  
db.inventoryA.find()
```

// \$pop

```
var query= { }  
var operacion= { $pop: { "tag": 1 } }  
db.inventoryA.updateMany(query,operacion)  
db.inventoryA.find()
```

// \$pull

```
var query= { }  
var operacion= { $addToSet: { "tag": "borrar" } }  
db.inventoryA.updateMany(query,operacion)  
var operacion= { $pull: { "tag": "borrar" } }  
db.inventoryA.updateMany(query,operacion)  
db.inventoryA.find()
```

// \$push

```
var query= { }  
var operacion= { $push: { "tag": "borrar" } }  
db.inventoryA.updateMany(query,operacion)  
db.inventoryA.find()
```

// \$each

```
var query= { }  
var operacion= { $push: { "tag": { $each: ["borrar","borrar"] } } }  
db.inventoryA.updateMany(query,operacion)  
db.inventoryA.find()
```

var query= { }

var operacion= { \$pull: { "tag": "borrar" } }

```
db.inventoryA.updateMany(query,operacion)
db.inventoryA.find()
```

```
// sort
var opcion= { "qty" : -1 }
db.inventory.find().sort(opcion)
```

```
// limit
var opcion= { "qty" : -1 }
db.inventory.find().sort(opcion).limit(1)
```

```
// count
db.inventory.find().count()
```

```
// toArray
db.inventory.find().toArray()
```

```
// Agregación
// ETAPAS
db.inventoryE.find()
```

```
// match
var query1 = { "item": "camiseta" }
var fase1 = { $match: query1 }
var etapas = [ fase1 ]
db.inventoryE.aggregate( etapas )
```

```
// project and addFields
var query1 = { "item": "camiseta" }
var fase1 = { $match: query1 }
var query2 = { "_id": 0, "elemento": "borrar" }
var fase2 = { $project: query2 }
var etapas = [ fase1, fase2 ]
db.inventoryE.aggregate( etapas )
```

```
var fase2 = { $addFields: query2 }
var etapas = [ fase1, fase2 ]
db.inventoryE.aggregate( etapas )
```

```
// group
var query1 = { "_id": "$item", "numero": { $sum: 1 } }
var fase1 = { $group: query1 }
var etapas = [ fase1 ]
db.inventoryE.aggregate( etapas )
```

```
// sort
var query1 = { "_id": "$item", "numero": { $sum: 1 } }
var fase1 = { $group: query1 }
```

```
var query2 = { "numero": -1 }
var fase2 = { $sort: query2 }
var etapas = [ fase1, fase2 ]
db.inventoryE.aggregate( etapas )
```

```
// limit
var query1 = { "_id": "$item", "numero": { $sum: 1 } }
var fase1 = { $group: query1 }
var query2 = { "numero": -1 }
var fase2 = { $sort: query2 }
var fase3 = { $limit: 1 }
var etapas = [ fase1, fase2, fase3 ]
db.inventoryE.aggregate( etapas )
```

```
// unwind
db.inventoryAE.find()
var fase1 = { $unwind: "$size" }
var etapas = [ fase1 ]
db.inventoryAE.aggregate( etapas )
```

```
// out
var fase1 = { $unwind: "$size" }
var query2 = { "_id": 0 }
var fase2 = { $project: query2 }
var fase3 = { $out: "borrar" }
var etapas = [ fase1, fase2, fase3 ]
db.inventoryAE.aggregate( etapas )
```

```
// Filtrar por camisetas,
// añadir nuevo campo importe total (qty * price)
// contar cuantas camisetas hay
// la suma de sus importes total
// el mínimo de price
// el máximo de price
var db.inventoryE.find()
var query1 = { item: "camiseta" }
var fase1 = { $match: query1 }
var subquery2 = { $multiply: ["$qty", "$price"] }
var query2 = { "importe_total": subquery2 }
var fase2 = { $addFields: query2 }
var query3 = { "_id": "$item", "cont": { $sum: 1 }, "total": { $sum: "$importe_total" }, "minimo": {
  $min: "$price" }, "maximo": { $max: "$price" } }
var fase3 = { $group: query3 }
var etapas = [ fase1 , fase2, fase3 ]
db.inventoryE.aggregate( etapas )
```

```
// Agrupar por item y añadir un array de documentos que tengan cada qty y price asociado
var query1 = { "_id": "$item", "cantidades": { $push: { "qty": "$qty", "price": "$price" } } }
```



```
var fase1 = { $group: query1 }
var etapas = [ fase1 ]
db.inventoryE.aggregate( etapas )
```

```
// Agrupar por item y tag (transformando a mayúsculas) y contar documentos, ordenar de
forma descendente por el conteo
var query1 = { "_id": { "item": { $toUpper: "$item" }, "tag": { $toUpper: "$tag" } }, tot: { $sum: 1
}}
var fase1 = { $group: query1 }
var query2 = { "tot": -1 }
var fase2 = { $sort: query2 }
var etapas = [ fase1, fase2 ]
db.inventoryE.aggregate( etapas )
```

```
// Insertar varios documentos
```

```
var array_items = [{ "title" : "The Pillars of Society", "artist" : "Grosz", "year" : 1926, "price" :
NumberDecimal("199.99") }, { "title" : "Melancholy III", "artist" : "Munch", "year" : 1902,
"price" : NumberDecimal("280.00") }, { "title" : "Dancer", "artist" : "Miro", "year" : 1925,
"price" : NumberDecimal("76.04") }, { "title" : "The Great Wave off Kanagawa", "artist" :
"Hokusai", "price" : NumberDecimal("167.30") }, { "title" : "The Persistence of Memory",
"artist" : "Dali", "year" : 1931, "price" : NumberDecimal("483.00") }, { "title" : "Composition
VII", "artist" : "Kandinsky", "year" : 1913, "price" : NumberDecimal("385.00") }, { "title" : "The
Scream", "artist" : "Munch", "year" : 1893 }, { "title" : "Blue Flower", "artist" : "O'Keefe", "year"
: 1918, "price" : NumberDecimal("118.42") } ]
```

```
db.arts.insertMany(array_items)
```

```
//1. Buscar Cuadros con año por encima de 1920
```

```
// select * from ejemplo.arts where year > 1920
```

```
var query={ year : { $gt : 1920}}
```

```
db.arts.find(query)
```

```
//2. Buscar Cuadros con año por encima de 1920 Y precio mayor a 200
```

```
// select * from ejemplo.arts where year > 1920 AND price > 200
```

```
var query={ year : { $gt : 1920}, price : { $gt : 200 } }
```

```
db.arts.find(query)
```

```
//3. Buscar Cuadros con año por encima de 1920 O precio mayor a 200
```

```
// select * from ejemplo.arts where year > 1920 OR price > 200
```

```
var filtro1={ year : { $gt : 1920} }
```

```
var filtro2= { price : { $gt : 200 } }
```

```
var query={ $or: [ filtro1, filtro2 ] }
```

```
db.arts.find(query)
```

```
//4. Buscar Cuadros que NO sean menores de 1925 y NO sean mayores de 1950
```

```
// select * from ejemplo.arts where NOT(year < 1925 AND year > 1950)
```

```
var filtro1={ year : { $lt : 1925} }
```

```
var filtro2= { year : { $gt : 1950 } }
```

```
var query={ $nor: [ filtro1, filtro2 ] }
```

```
db.arts.find(query)
```

```
//5. Buscar Cuadros que NO ESTÉN DATADOS (no tengan el campo year)
```

```
db.arts.find({ year : { $exists : false} })
```

```
//6. Añadir campo year igual a NULL en campos que no estén datados
```

```
db.arts.updateMany({ year : { $exists : false} }, { $set: { "year" : null } })
```

```
// YA NO DEVUELVE DOCUMENTOS QUE NO TIENEN CAMPO YEAR PORQUE TODOS LO TIENEN
```

```
db.arts.find({ year : { $exists : false} })
```

```
//7. Ordenar por año de forma descendente
```

```
// select * from ejemplo.arts order by year DESC
```

```
db.arts.find({}).sort({ year: -1})
```

```
//8. Limitar la salida anterior a las 3 primeras filas
```

```
db.arts.find({}).sort({ year: -1}).limit(3)
```

```
//9. Sacar las filas de la cuarta a la sexta
```

```
db.arts.find({}).sort({ year: -1}).skip(3).limit(3)
```

dataset

<https://drive.google.com/file/d/1Xyxa1F87xwVszBtZsbZkSC31riGs-WR7/view?usp=sharing>

//1. Filtrar por cuisine American

```
var query = { "cuisine": "American" }
```

```
db.dataset.find(query)
```

//2. Filtrar por cuisine American y mostrar campo borough

```
var query = { "cuisine": "American" }
```

```
var project1 = { "borough": 1 }
```

```
db.dataset.find(query,project1)
```

//3. Filtrar por cuisine American, mostrar campo borough y quitar _id

```
var query = { "cuisine": "American" }
```

```
var project2 = { "borough": 1, '_id': 0 }
```

```
db.dataset.find(query,project2)
```

//4. Filtrar por cuisine American, que no salga campo zipcode y quitar _id

```
var query = { "cuisine": "American" }
```

```
var project3 = { "zipcode": 0, '_id': 0 }
```

```
db.dataset.find(query,project3)
```

//5. Filtrar por cuisine American, después contar número de documentos y después quitamos el campo _id

```
// select count(1) from dataset where cuisine = 'American'
```

```
var fase1= { "$match" : { "cuisine": "American" }}  
var fase2= { "$group" : { "_id": null, "count" : { "$sum" : 1} }}  
var fase3= { "$project": { "_id" : 0 } }  
var etapas= [ fase1, fase2, fase3 ]  
db.dataset.aggregate( etapas )
```

//6. Media de puntuaciones de restaurante de cuisine Italian

```
var fase1= { $match: { "cuisine": "Italian" } }  
var fase2= { $group: { "_id": null, "media": { "$avg" : "$score" }}}  
var etapas= [ fase1, fase2 ]  
db.dataset.aggregate( etapas )
```

//7. Media de puntuaciones de restaurante de cuisine Italian por borough

```
var fase1= { $match: { "cuisine": "Italian" } }  
var fase2= { $group: { "_id": "$borough", "media": { "$avg" : "$score" } } }  
var etapas= [ fase1, fase2 ]  
db.dataset.aggregate( etapas )
```

//8. Media de puntuaciones de restaurante por borough y cuisine ordenado de mayor a menor puntuación de media y que su media se mayor a 12

```
var fase1= { $group: { "_id": { "barrio": "$borough", "cocina": "$cuisine" }, "media": { "$avg" : "$score" } } }  
var fase2= { $sort: { "media" : -1 } }  
var fase3= { $match: { "media" : { $gt: 12 } } }  
var etapas= [ fase1, fase2, fase3 ]  
db.dataset.aggregate( etapas )
```

//8bis. Calcular la media de puntuaciones de restaurante por borough y cuisine y la cantidad de documentos de cada uno de ellos, ordenado de mayor a menor puntuación de media, después que su media se mayor a 12 y que al menos tengan 20 documentos (valoraciones). Comparar con el resultado anterior.

```
var fase1= { $group: { "_id": { "barrio": "$borough" , "cocina": "$cuisine" }, "media": { "$avg" : "$score" }, "count": { "$sum" : 1 } } }

var fase2= { $sort: { "media" : -1 } }

var fase3= { $match: { "media" : { $gt: 12 }, "count" : { $gte: 20 } } }

var etapas= [ fase1, fase2, fase3 ]

db.dataset.aggregate( etapas )
```

//9. Agregar por grade y sumar cuantos tiene cada uno, ordenando de forma descendente

```
var fase1= { $group: { "_id": "$grade", "cuenta": { $sum: 1 } } }

var fase2= { $sort: { "cuenta" : -1 } }

var etapas= [ fase1, fase2 ]

db.dataset.aggregate( etapas )
```

//10. WHERE -> FILTRAR POR COCINA AMERICANA , GRADO A Y PUNTUACIÓN ENTRE 8 y 12, (MAYOR MENOR IGUAL)

// GROUP -> AGRUPANDO POR BARRIO,

// CALCULO -> OBTENER EL NÚMERO DE DOCUMENTOS Y LA MEDIA, EL MÁXIMO Y MÍNIMO DE PUNTUACIONES

// ORDENAR -> POR MEDIA DE FORMA DESCENDENTE

```
var filtro1 = { "cuisine" : "American" }

var filtro2 = { "grade": "A" }

var filtro3 = { "score": { $gte: 8 , $lte: 12 } }

var fase1= { $match: { $and: [ filtro1, filtro2, filtro3 ] } }
```

```
var fase2= { $group: { "_id": "$borough", "conteo": { $sum: 1 }, "media": { $avg: "$score" },  
"max": { $max: "$score" }, "min": { $min: "$score" } } }
```

```
var fase3= { $sort: { "media": -1 } }
```

```
var etapas= [ fase1, fase2, fase3 ]
```

```
db.dataset.aggregate( etapas )
```

//11. Concatenar calle y código postal en mayúsculas en un nuevo campo

```
var concatenar = { $concat: [ "$street", " ", "$zipcode" ] }
```

```
var direccion = { $toUpper: concatenar }
```

```
var fase1 = { $project: { _id: 0, "direccion" : direccion } }
```

```
var etapas = [ fase1 ]
```

```
db.dataset.aggregate( etapas )
```

Subconsultas

```
db.reportedsales.insertMany( [  
  { quarter: "2019Q1", region: "A", qty: 400 },  
  { quarter: "2019Q1", region: "B", qty: 550 },  
  { quarter: "2019Q1", region: "C", qty: 1000 },  
  { quarter: "2019Q2", region: "A", qty: 660 },  
  { quarter: "2019Q2", region: "B", qty: 500 },  
  { quarter: "2019Q2", region: "C", qty: 1200 }  
)
```

```
var docu = { region: "$region", qty: "$qty" }
```

```
var fase1 = { $group: { _id: "$quarter", tot: { $sum: "$qty" }, items: { $push: docu } } }
```

```
var pipeline = [ fase1 ]
```

```
db.reportedsales.aggregate( pipeline )
```

```
var fase2 = { $unwind: "$items" }
```

```
var pipeline = [ fase1, fase2 ]
```

```
db.reportedsales.aggregate( pipeline )
```

```
var merge = [ { quarter: "$_id", tot: "$tot" }, "$items" ]
```

```
var fase3 = { $replaceWith: { $mergeObjects: merge } }
```

```
var pipeline = [ fase1, fase2, fase3 ]
```

```
db.reportedsales.aggregate( pipeline )
```

```
var fase4 = { $addFields: { "%region": { "$multiply": [{" $divide": [{" $qty", "$tot" }], 100 } } } }
var pipeline = [ fase1, fase2, fase3, fase4 ]
db.reportedsales.aggregate( pipeline )
```

Veréis cómo obtenemos el resultado correcto. Os pongo a continuación la docu de cada uno de los operadores utilizados para ampliar vuestro conocimiento sobre ellos:

```
$group -> https://docs.mongodb.com/manual/reference/operator/aggregation/group/
$push -> https://docs.mongodb.com/manual/reference/operator/aggregation/push/
$sum -> https://docs.mongodb.com/manual/reference/operator/aggregation/sum/
$unwind -> https://docs.mongodb.com/manual/reference/operator/aggregation/unwind/
$mergeObjects ->
https://docs.mongodb.com/manual/reference/operator/aggregation/mergeObjects
$replaceWith ->
https://docs.mongodb.com/manual/reference/operator/aggregation/replaceWith/
$addFields -> https://docs.mongodb.com/manual/reference/operator/aggregation/addFields/
$multiply -> https://docs.mongodb.com/manual/reference/operator/aggregation/multiply/
$divide -> https://docs.mongodb.com/manual/reference/operator/aggregation/divide/
```

Fechas

```
var a = { fecha: "2 January 2018" }
db.prueba.insert(a)
```

```
db.prueba.find({})
```

```
db.prueba.aggregate([
  { $project: { fecha: 1, nuevo: { $toDate: "$fecha" } } }
])
```

```
var array_fechas = [ { "fecha": "2018-01-02" }, { "fecha": "2018-01-03" }, { "fecha":
"2019-01-03" }, { "fecha": "2018-02-01" } ]
db.fechasejemplo.insertMany(array_fechas)
db.fechasejemplo.find()
```

```
var anio = { $year: { $toDate: '$fecha' } }
var mes = { $month: { $toDate: '$fecha' } }
var fecha = { 'fechanueva': { 'year': anio, 'month': mes } }
var fase1 = { $addFields: fecha }
var etapas = [ fase1 ]
db.fechasejemplo.aggregate( etapas )
```

```
var filtro = { 'fechanueva.year' : { $eq: 2018 }, 'fechanueva.month' : { $eq: 1 } }
var fase2 = { $match: filtro }
var etapas = [ fase1, fase2 ]
```

```
db.fechasejemplo.aggregate( etapas )
```

```
// EJEMPLOS FECHAS
```

```
a = [ { qty: 1, fecha: "2019-01-01 10:30:02" }, { qty: 3, fecha: "2019-02-05 10:30:02" }, { qty: 3, fecha: "2019-01-05 10:30:02" }, { qty: 2, fecha: "2018-01-01 10:30:02" } ]
```

```
db.fecha.insertMany(a)
```

```
db.fecha.find()
```

```
// TRANSFORMAR FECHA
```

```
fase1={ $addFields: { fecha_date: { $toDate: "$fecha" } } }
```

```
pipeline=[ fase1 ]
```

```
db.fecha.aggregate( pipeline )
```

```
// Diferencia entre fechas (restar dos fechas).
```

```
fase1={ $addFields: { fecha_date: { $toDate: "$fecha" } } }
```

```
fase2={ $addFields: { fecha_diff: { $dateDiff: { startDate: new Date(), endDate: "$fecha_date", unit: "day" } } } }
```

```
pipeline=[ fase1, fase2 ]
```

```
db.fecha.aggregate( pipeline )
```

```
// Restar tiempo a una fecha.
```

```
fase1={ $addFields: { fecha_date: { $toDate: "$fecha" } } }
```

```
fase2={ $addFields: { fecha_diff: { $dateSubtract: { startDate: "$fecha_date", unit: "day", amount: 2 } } } }
```

```
pipeline=[ fase1, fase2 ]
```

```
db.fecha.aggregate( pipeline )
```

```
// Cómo agregar por año a partir de una fecha completa
```

```
fase1={ $addFields: { fecha_date: { $toDate: "$fecha" } } }
```

```
fase2={ $group: { "_id": { $year: "$fecha_date" }, suma: { $sum: "$qty" } } }
```

```
pipeline=[ fase1, fase2 ]
```

```
db.fecha.aggregate( pipeline )
```

```
// Cómo agregar por año y mes a partir de una fecha completa
```

```
fase1={ $addFields: { fecha_date: { $toDate: "$fecha" } } }
```

```
fase2={ $group: { "_id": { "year": { $year: "$fecha_date" }, "month": { $month: "$fecha_date" } }, suma: { $sum: "$qty" } } }
```

```
pipeline=[ fase1, fase2 ]
```

```
db.fecha.aggregate( pipeline )
```


//Os pongo la referencia de todos los comandos que he utilizado en los ejemplos:

<https://docs.mongodb.com/manual/reference/operator/aggregation/toDate/>
<https://docs.mongodb.com/manual/reference/operator/aggregation/year/>
<https://docs.mongodb.com/manual/reference/operator/aggregation/month/>
<https://docs.mongodb.com/manual/reference/operator/aggregation/divide/>
<https://docs.mongodb.com/manual/reference/operator/aggregation/subtract/>

Strings

```
var inserts=[{"precio": "-$100,56~$1000,23"}]  
db.precios.insertMany(inserts)
```

```
var prices = { $split: [ "$precio", "~" ] }
```

```
var fase1 = { $addField: { "precios": prices } }
```

```
var etapas = [ fase1 ]
```

```
db.precios.aggregate(etapas)
```

```
var prices = { $split: [ "$precio", "~" ] }  
var fase1 = { $addField: { "precios": prices } }
```

```
var fase2= { $unwind: "$precios" }  
var fase3= { $set: { "preciolimpio": { $replaceAll: { input: "$precios", find: { $literal: '$' },  
replacement: "" } } } }  
var etapas=[fase1,fase2, fase3]  
db.precios.aggregate(etapas)
```

```
var prices = { $split: [ "$precio", "~" ] }  
var fase1 = { $addField: { "precios": prices } }
```

```
var fase2= { $unwind: "$precios" }  
var fase3= { $set: { "preciolimpio": { $replaceAll: { input: "$precios", find: { $literal: '$' },  
replacement: "" } } } }  
var fase4={ $set: { "conpunto": { $replaceAll: { input: "$preciolimpio", find: ",", replacement:  
"." } } } }  
var etapas=[fase1,fase2, fase3, fase4]  
db.precios.aggregate(etapas)
```

```
var prices = { $split: [ "$precio", "~" ] }  
var fase1 = { $addField: { "precios": prices } }
```

```
var fase2= { $unwind: "$precios" }
var fase3= { $set: { "preciolimpio": { $replaceAll: { input: "$precios", find: { $literal: '$' },
replacement: "" } } } }
var fase4={ $set: { "conpunto": { $replaceAll: { input: "$preciolimpio", find: ",", replacement:
"." } } } }
var fase5={ $set: { "numero": { $toDecimal: "$conpunto" } } }
var etapas=[fase1,fase2, fase3, fase4, fase5]
db.precios.aggregate(etapas)
```